Summarize

Chapter 6. Workflow

In the previous two chapters, we've looked at aspects of microservices related to how one microservice talks to another. But what happens when we want multiple microservices to collaborate, perhaps to implement a business process? How we model and implement these sorts of workflows in distributed systems can be a tricky thing to get right.

In this chapter, we'll look at the pitfalls associated with using distributed transactions to solve this problem, and we'll also look at sagas—a concept that can help us model our microservice workflows in a much more satisfactory manner.

Database Transactions

Generically speaking, when we think of a transaction in the context of computing, we think of one or more actions that are going to occur that we want to treat as a single unit. When making multiple changes as part of the same overall operation, we want to confirm whether all the changes have been made. We also want a way to clean up after ourselves if an error occurs while these changes are happening. Typically, this results in us using something like a database transaction.

With a database, we use a transaction to ensure that one or more state changes have been made successfully. This could include data being removed, inserted, or changed. In a relational database, this could involve multiple tables being updated within a single transaction.

ACID Transactions

Typically when we talk about database transactions, we are talking about ACID transactions. ACID is an acronym outlining the key properties of database transactions that lead to a system we can rely on to ensure the durability and consistency of our data storage.

ACID stands for *atomicity*, *consistency*, *isolation*, *and durability*, and here is what these properties give us:

Atomicity

Ensures that the operations attempted within the transaction either all complete or all fail. If any of the changes we're trying to make fail for some reason, then the whole operation is aborted, and it's as though no changes were ever made.

Consistency

When changes are made to our database, we ensure it is left in a valid, consistent state.

Isolation

Allows multiple transactions to operate at the same time without interfering. This is achieved by ensuring that any interim state changes made during one transaction are invisible to other transactions.

Durability

Makes sure that once a transaction has been completed, we are confident the data won't get lost in the event of some system failure.

It's worth noting that not all databases provide ACID transactions. All relational database systems I've ever used do, as do many of the newer NoSQL databases like Neo4j. MongoDB for many years supported ACID transactions only on changes being made to a single document, which could cause issues if you wanted to make an atomic update to more than one document.[1]

This isn't the book for a detailed exploration into these concepts; I've certainly simplified some of these descriptions for the sake of brevity. For those who would like to explore these concepts further, I recommend *Designing Data-Intensive Applications*.[2] We'll mostly concern ourselves with atomicity in what follows. That's not to say that the other properties aren't also important, but grappling with how to deal with the atomicity of database operations tends to be the first issue we hit when we start breaking apart functionality into microservices.

Still ACID, but Lacking Atomicity?

I want to be clear that we can still use ACID-style transactions when using microservices. A microservice is free to use an ACID transaction for operations to its own database, for

example. It's just that the scope of these transactions is reduced to state change that happens locally within that single microservice. Consider Figure 6-1. Here, we are keeping track of the process involved in onboarding a new customer to MusicCorp. We've reached the end of the process, which involves changing the `Status` of customer `2346` from `PENDING` to `VERIFIED`. As the enrollment is now complete, we also want to remove the matching row from the `PendingEnrollments` table. With a single database, this is done in the scope of a single ACID database transaction—either both of these state changes occur, or neither occurs.

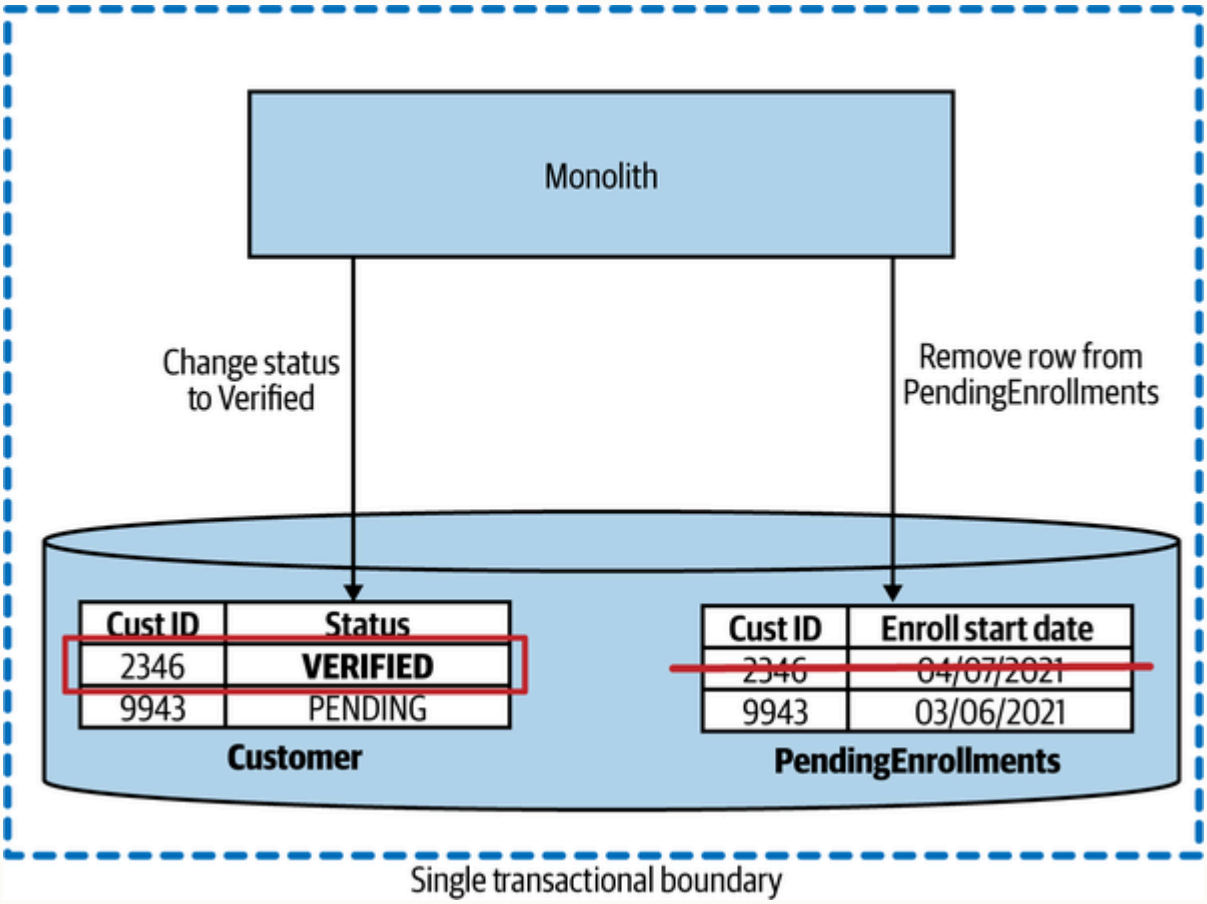


Figure 6-1. Updating two tables in the scope of a single ACID transaction

Compare this with Figure 6-2, where we're making exactly the same change, but each change is made in a different database. This means there are two transactions to consider, each of which could work or fail independently of the other.

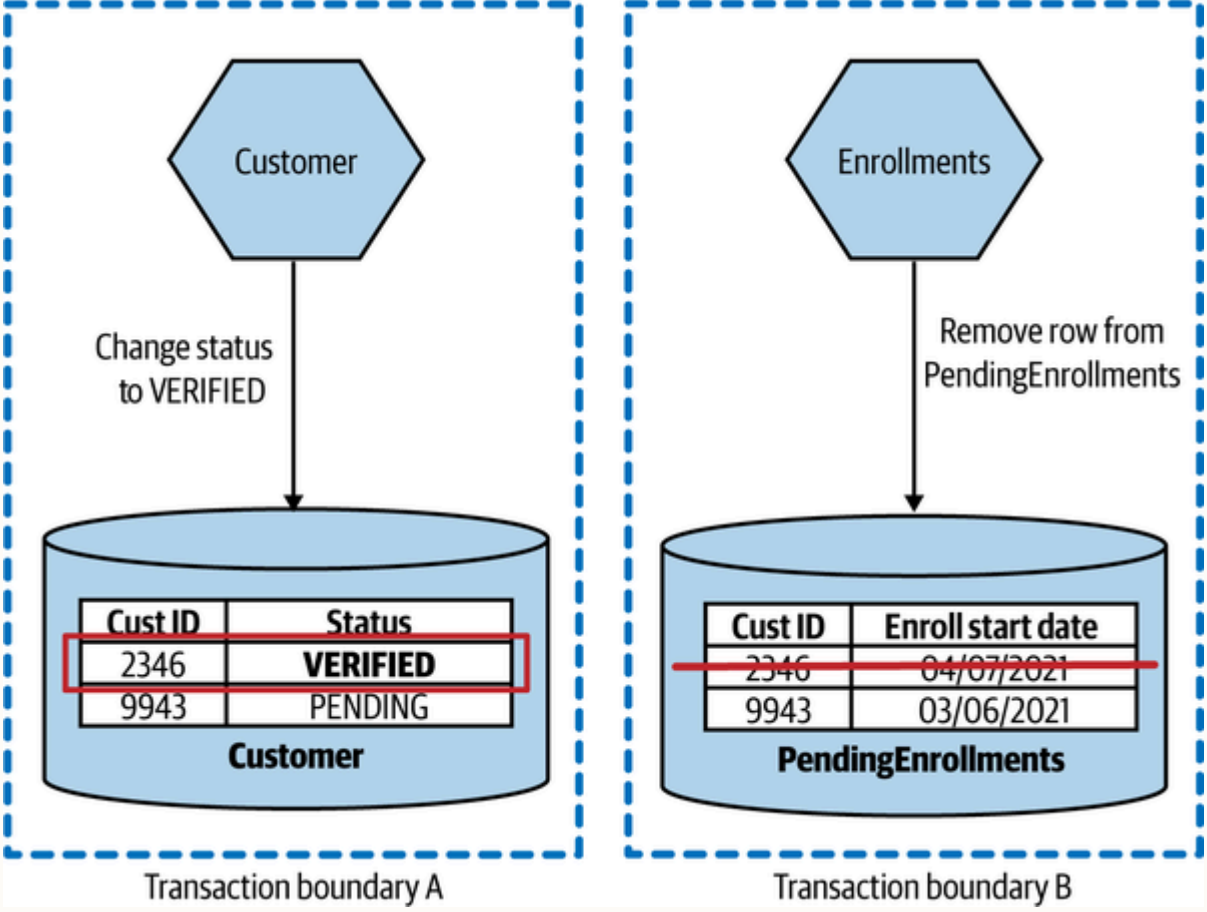


Figure 6-2. Changes made by the `Customer` and `Enrollments` microservices are now done in the scope of two different transactions

We could decide to sequence these two transactions, of course, removing a row from the `PendingEnrollments` table only if we could change the row in the `Customer` table. But we'd still have to reason about what to do if the deletion from the `PendingEnrollments` table then failed—all logic that we'd need to implement ourselves. Being able to reorder steps to better handle these use cases can be a really useful idea, though (one we'll come back to when we explore sagas). But fundamentally, we have to accept that by decomposing this operation into two separate database transactions, we've lost guaranteed atomicity of the operation as a whole.

This lack of atomicity can start to cause significant problems, especially if we are migrating systems that previously relied on this property. Normally, the first option that people start considering is still using a single transaction, but one that now spans multiple processes—a distributed transaction. Unfortunately, as we'll see, distributed transactions may not be the right way forward. Let's look at one of the most common algorithms for implementing distributed transactions, the two-phase commit, as a way of exploring the challenges associated with distributed transactions as a whole.

Distributed Transactions—Two-Phase Commits

The *two-phase commit algorithm* (sometimes shortened to *2PC*) is frequently used in an attempt to give us the ability to make transactional changes in a distributed system, where multiple separate processes may need to be updated as part of the overall operation. Distributed transactions, and two-phased commits more specifically, are frequently considered by teams moving to microservice architectures as a way of solving challenges they face. But as we'll see, they might not solve your problems and may bring even more confusion to your system.

The 2PC is broken into two phases (hence the name *two-phase commit*): a voting phase and a commit phase. During the *voting phase*, a central coordinator contacts all the workers who are going to be part of the transaction and asks for confirmation as to whether or not some state change can be made. In Figure 6-3, we see two requests: one to change a customer status to `VERIFIED`, and another to remove a row from our `PendingEnrollments` table. If all workers agree that the state change they are asked for can take place, the algorithm proceeds to the next phase. If any worker says the change cannot take place, perhaps because the requested state change violates some local condition, the entire operation aborts.

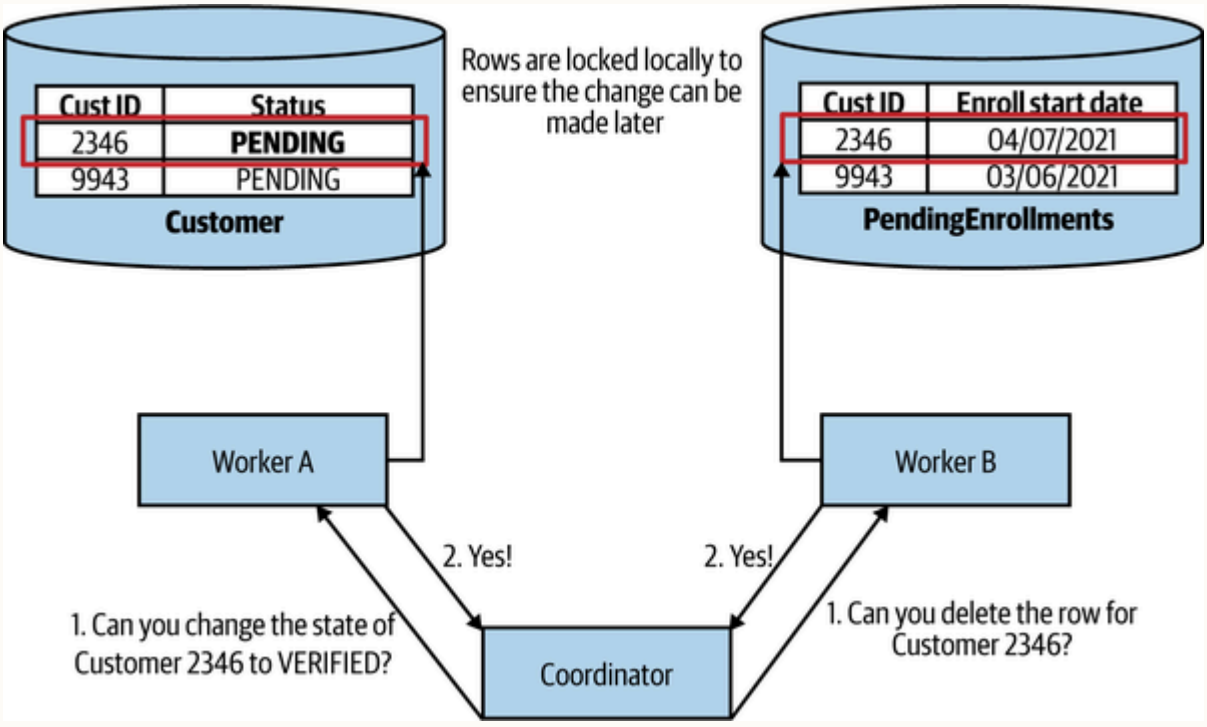


Figure 6-3. In the first phase of a two-phase commit, workers vote to decide if they can carry out some local state change

It's important to highlight that the change does not take effect immediately after a worker indicates that it can make the change. Instead, the worker is guaranteeing that it will be able to make that change at some point in the future. How would the worker

make such a guarantee? In Figure 6-3, for example, `Worker A` has said it will be able to change the state of the row in the `Customer` table to update that specific customer's status to `VERIFIED`. What if a different operation at some later point deletes the row, or makes some other smaller change that nonetheless means that a change to `VERIFIED` later is invalid? To guarantee that the change to `VERIFIED` can be made later, `Worker A` will likely have to lock the record to ensure that other changes cannot take place.

If any workers didn't vote in favor of the commit, a rollback message needs to be sent to all parties to ensure that they can clean up locally, which allows the workers to release any locks they may be holding. If all workers agreed to make the change, we move to the commit phase, as in Figure 6-4. Here, the changes are actually made, and associated locks are released.

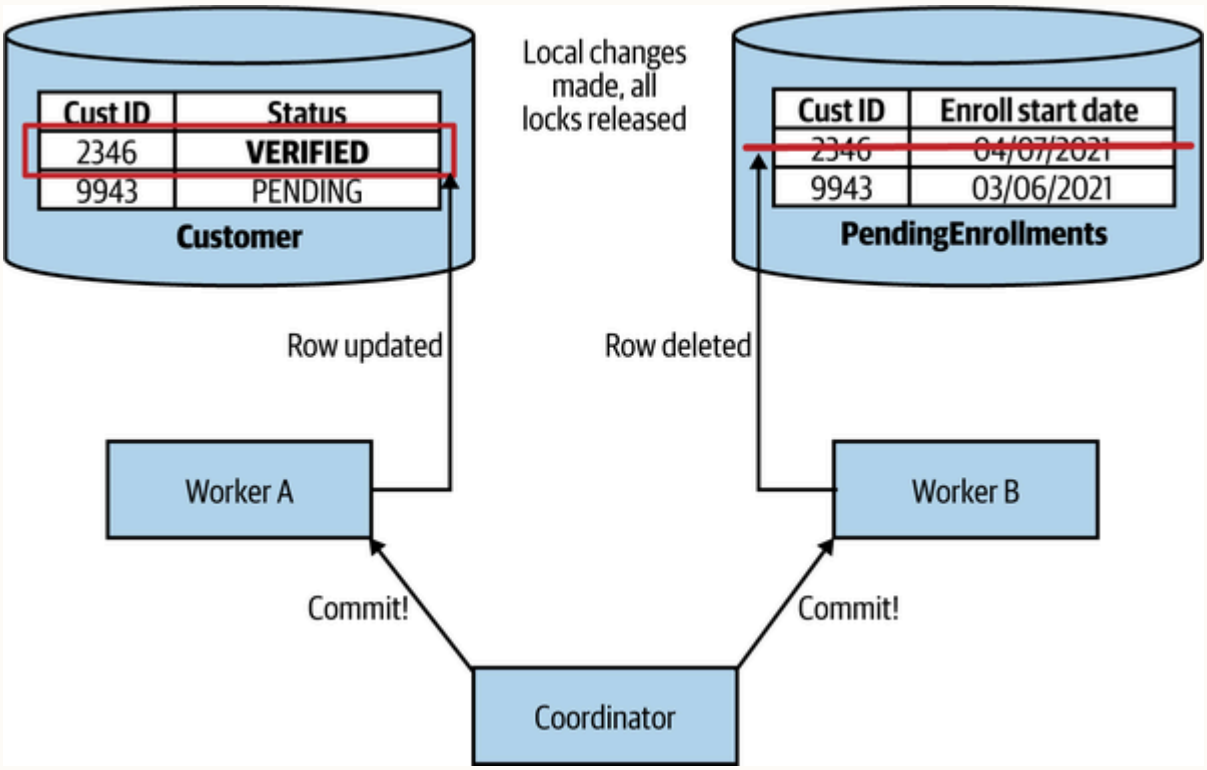


Figure 6-4. In the commit phase of a two-phase commit, changes are actually applied

It's important to note that in such a system, we cannot in any way guarantee that these commits will occur at exactly the same time. The `Coordinator` needs to send the commit request to all participants, and that message could arrive and be processed at different times. This means that it's possible we could see the change made to `Worker A` but not yet to `Worker B`, if we could directly observe the state of either worker. The more latency there is between the `Coordinator` and the participants in the two-phase commit, and the more slowly the workers process the response, the wider this window of inconsistency might be. Coming back to our definition of ACID, *isolation* ensures that we don't see intermediate states during a transaction. But with this two-phase commit, we've lost that guarantee.

When a two-phase commit works, at its heart it is very often just coordinating distributed locks. The workers need to lock local resources to ensure that the commit can take place during the second phase. Managing locks and avoiding deadlocks in a single-process system isn't fun. Now imagine the challenges of coordinating locks among multiple participants. It's not pretty.

There are a host of failure modes associated with two-phase commits that we don't have time to explore. Consider the problem of a worker voting to proceed with the transaction but then not responding when asked to commit. What should we do then? Some of these failure modes can be handled automatically, but some can leave the system in such a state that things need to be fixed manually by an operator.

The more participants you have, and the more latency you have in the system, the more issues a two-phase commit will have. 2PC can be a quick way to inject huge amounts of latency into your system, especially if the scope of locking is large, or if the duration of the transaction is large. It's for this reason two-phase commits are typically used only for very short-lived operations. The longer the operation takes, the longer you've got resources locked!

Distributed Transactions—Just Say No

For all the reasons outlined so far, I strongly suggest you avoid the use of distributed transactions like the two-phase commit to coordinate changes in state across your microservices. So what else can you do?

Well, the first option could be to just not split the data apart in the first place. If you have pieces of state that you want to manage in a truly atomic and consistent way, and you cannot work out how to sensibly get these characteristics without an ACID-style transaction, then leave that state in a single database, and leave the functionality that manages that state in a single service (or in your monolith). If you're in the process of working out where to split your monolith and what decompositions might be easy (or hard), then you could well decide that splitting apart data that is currently managed in a transaction is just too difficult to handle right now. Work on some other area of the system, and come back to this later.

But what happens if you really do need to break this data apart, but you don't want all the pain of managing distributed transactions? How can you carry out operations in multiple services but avoid locking? What if the operation is going to take minutes, days, or

perhaps even months? In cases like this, you may consider an alternative approach: sagas.

DATABASE DISTRIBUTED TRANSACTIONS

I am arguing against the general use of distributed transactions to coordinate state change across microservices. In such situations, each microservice is managing its own local durable state (e.g., in its database). Distributed transactional algorithms are being used successfully for some large-scale databases, Google's Spanner being one such system. In this situation, the distributed transaction is being applied transparently from an application's point of view by the underling database, and the distributed transaction is just being used to coordinate state changes within a single logical database (albeit one that might be distributed across multiple machines, and potentially across multiple data centers).

What Google has managed to achieve with Spanner is impressive, but it's also worth noting that what it had to do to make this work gives you an idea of the challenges involved. Let's just say it involves very expensive data centers and satellite-based atomic clocks (really). For a good overview of how Spanner makes this work, I recommend the presentation "Google Cloud Spanner: Global Consistency at Scale."[3]

Sagas

Unlike a two-phase commit, a *saga* is by design an algorithm that can coordinate multiple changes in state, but avoids the need for locking resources for long periods of time. A saga does this by modeling the steps involved as discrete activities that can be executed independently. Using sagas comes with the added benefit of forcing us to explicitly model our business processes, which can have significant benefits.

The core idea, first outlined in "Sagas" by Hector Garcia-Molina and Kenneth Salem,[4] addresses how best to handle operations known as *long lived transactions* (LLTs). These transactions might take a long time (minutes, hours, or perhaps even days) and as part of that process require changes to be made to a database.

If you directly mapped an LLT to a normal database transaction, a single database transaction would span the entire life cycle of the LLT. This could result in multiple rows or even full tables being locked for long periods of time while the LLT is taking place, caus-

ing significant issues if other processes are trying to read or modify these locked resources.

Instead, the authors of the paper suggest we should break down these LLTs into a sequence of transactions, each of which can be handled independently. The idea is that the duration of each of these "sub" transactions will be shorter, and will modify only part of the data affected by the entire LLT. As a result, there will be far less contention in the underlying database as the scope and duration of locks is greatly reduced.

While sagas were originally envisaged as a mechanism to help with LLTs acting against a single database, the model works just as well for coordinating change across multiple services. We can break a single business process into a set of calls that will be made to collaborating services—this is what constitutes a saga.

NOTE

Before we go any further, you need to understand that a saga does *not* give us atomicity in ACID terms such as we are used to with a normal database transaction. As we break the LLT into individual transactions, we don't have atomicity at the level of the saga itself. We do have atomicity for each individual transaction inside the overall saga, as each one of them can relate to an ACID transactional change if needed. What a saga gives us is enough information to reason about which state it's in; it's up to us to handle the implications of this.

Let's take a look at a simple order fulfillment flow for MusicCorp, outlined in Figure 6-5, which we can use to further explore sagas in the context of a microservice architecture.

Here, the order fulfillment process is represented as a single saga, with each step in this flow representing an operation that can be carried out by a different service. Within each service, any state change can be handled within a local ACID transaction. For example, when we check and reserve stock using the `Warehouse` service, internally the `Warehouse` service might create a row in its local `Reservation` table recording the reservation; this change would be handled within a normal database transaction.

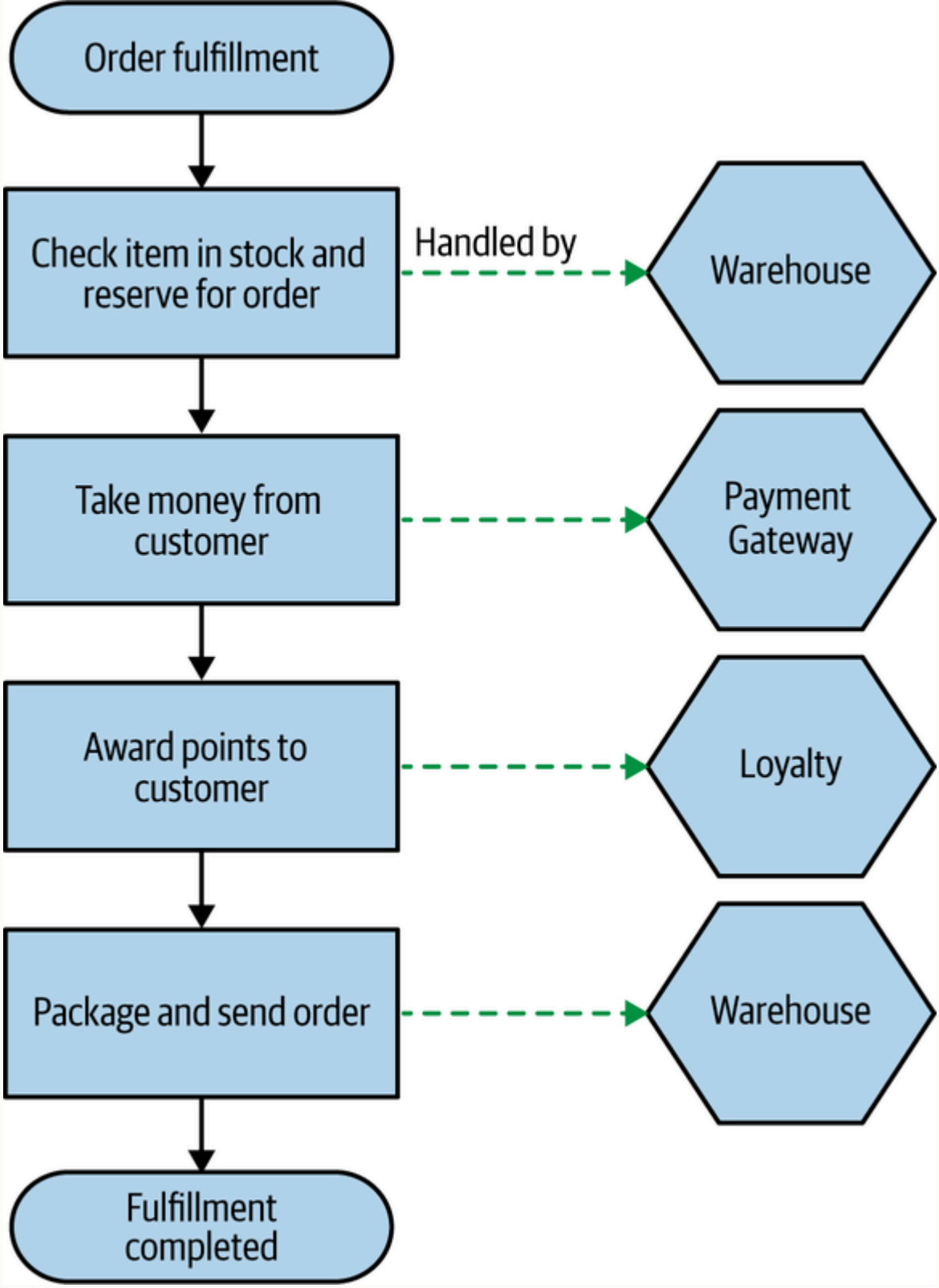


Figure 6-5. An example order fulfillment flow, along with the services responsible for carrying out the operation

Saga Failure Modes

With a saga being broken into individual transactions, we need to consider how to handle failure—or, more specifically, how to recover when a failure happens. The original saga paper describes two types of recovery: backward recovery and forward recovery.

Backward recovery involves reverting the failure and cleaning up afterwards—a rollback. For this to work, we need to define compensating actions that allow us to undo previously committed transactions. *Forward recovery* allows us to pick up from the point

where the failure occurred and keep processing. For that to work, we need to be able to retry transactions, which in turn implies that our system is persisting enough information to allow this retry to take place.

Depending on the nature of the business process being modeled, you may expect that any failure mode triggers a backward recovery, a forward recovery, or perhaps a mix of the two.

It's really important to note that a saga allows us to recover from *business* failures, not *technical* failures. For example, if we try and take payment from the customer but the customer has insufficient funds, then this is a business failure that the saga should be expected to handle. On the other hand, if the `Payment Gateway` times out or throws a `500 Internal Service Error`, then this is a technical failure that we need to handle separately. The saga assumes the underlying components are working properly—that the underlying system is reliable, and that we are then coordinating the work of reliable components. We'll explore some of the ways we can make our technical components more reliable in Chapter 12, but for more on this limitation of sagas, I recommend "The Limits of the Saga Pattern" by Uwe Friedrichsen.

Saga rollbacks

With an ACID transaction, if we hit a problem, we trigger a rollback before a commit occurs. After the rollback, it is like nothing ever happened: the change we were trying to make didn't take place. With our saga, though, we have multiple transactions involved, and some of those may have already committed before we decide to roll back the entire operation. So how can we roll back transactions after they have already been committed?

Let's come back to our example of processing an order, as outlined in Figure 6-5. Consider a potential failure mode. We've gotten as far as trying to package the item, only to find the item can't be found in the warehouse, as shown in Figure 6-6. Our system thinks the item exists, but it's just not on the shelf!

Now, let's assume that we decide we want to just roll back the entire order, rather than giving the customer the option for the item to be placed on back order. The problem is that we've already taken payment and awarded loyalty points for the order.

If these steps had all been done in a single database transaction, a simple rollback would clean it all up. However, each step in the order fulfillment process was handled by a different service call, each of which operated in a different transactional scope. There is no simple "rollback" for the entire operation.

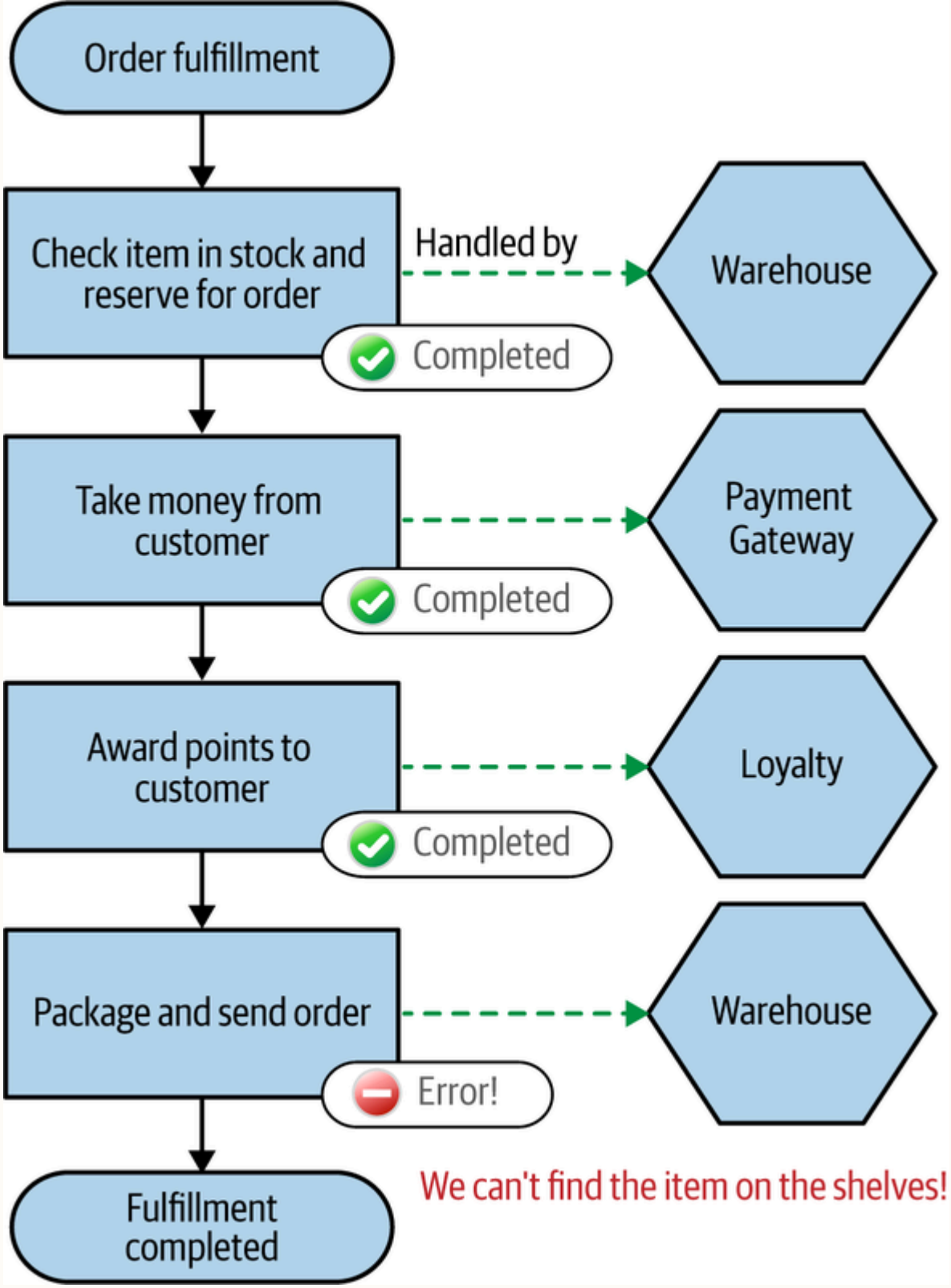


Figure 6-6. We've tried to package our item, but we can't find it in the warehouse

Instead, if you want to implement a rollback, you need to implement a compensating transaction. A *compensating transaction* is an operation that undoes a previously committed transaction. To roll back our order fulfillment process, we would trigger the compensating transaction for each step in our saga that has already been committed, as shown in Figure 6-7.

It's worth calling out the fact that these compensating transactions may not behave exactly as those of a normal database rollback. A database rollback happens before the commit, and after the rollback, it is as though the transaction never happened. In this

situation, of course, these transactions *did* happen. We are creating a new transaction that reverts the changes made by the original transaction, but we can't roll back time and make it as though the original transaction didn't occur.

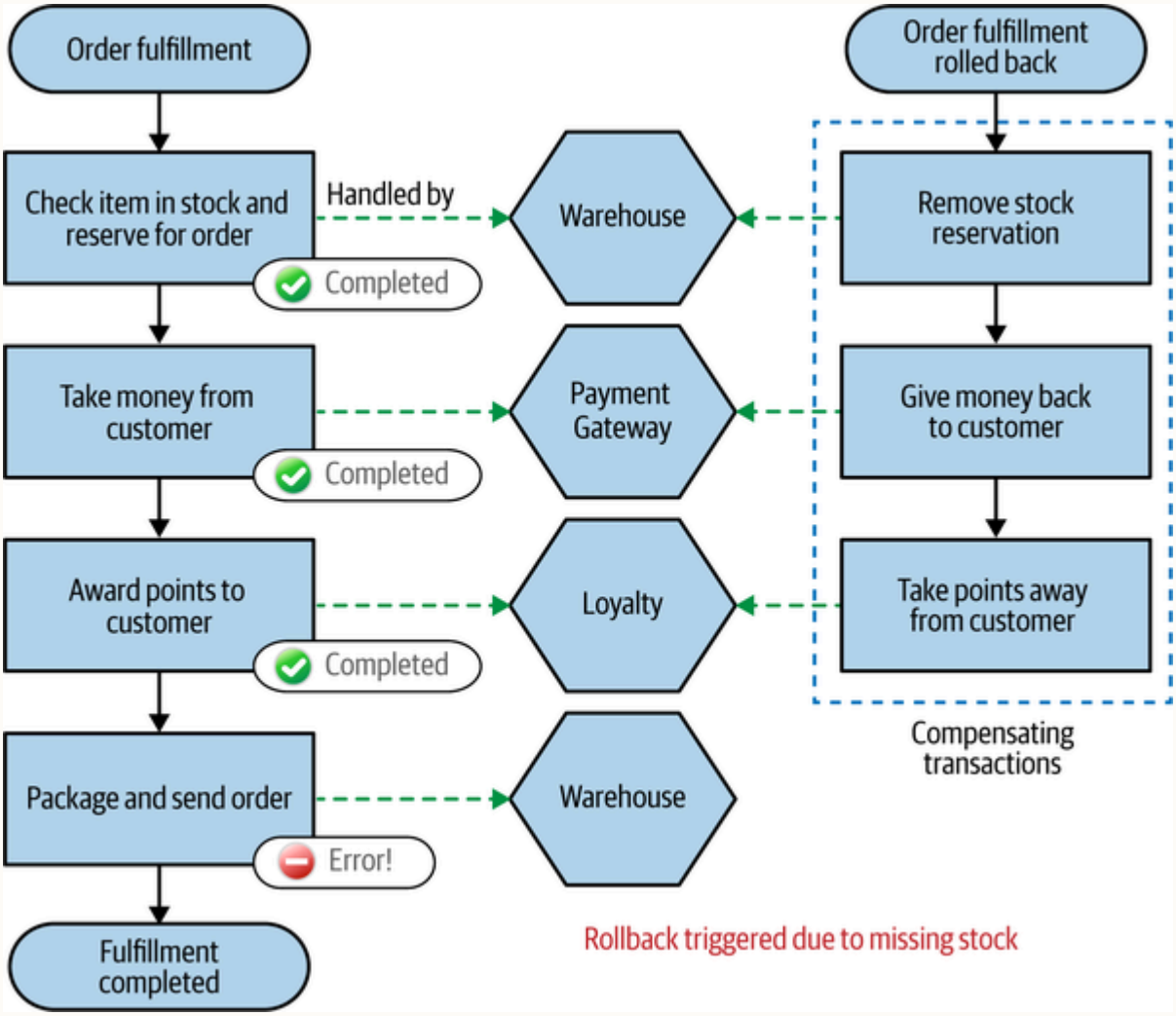


Figure 6-7. Triggering a rollback of the entire saga

Because we cannot always cleanly revert a transaction, we say that these compensating transactions are *semantic rollbacks*. We cannot always clean up everything, but we do enough for the context of our saga. As an example, one of our steps may have involved sending an email to a customer to tell them their order was on the way. If we decide to roll that back, we can't unsend an email![5] Instead, our compensating transaction could cause a second email to be sent to the customer, informing them that there was a problem with the order and it has canceled.

It is totally appropriate for information related to the rollback to persist in the system. In fact, this may be very important information. You may want to keep a record in the `Order` service for this aborted order, along with information about what happened, for a whole host of reasons.

Reordering workflow steps to reduce rollbacks

In Figure 6-7, we could have made our likely rollback scenarios somewhat simpler by re-ordering the steps in our original workflow. A simple change would be to award points only when the order is actually dispatched, as seen in Figure 6-8.

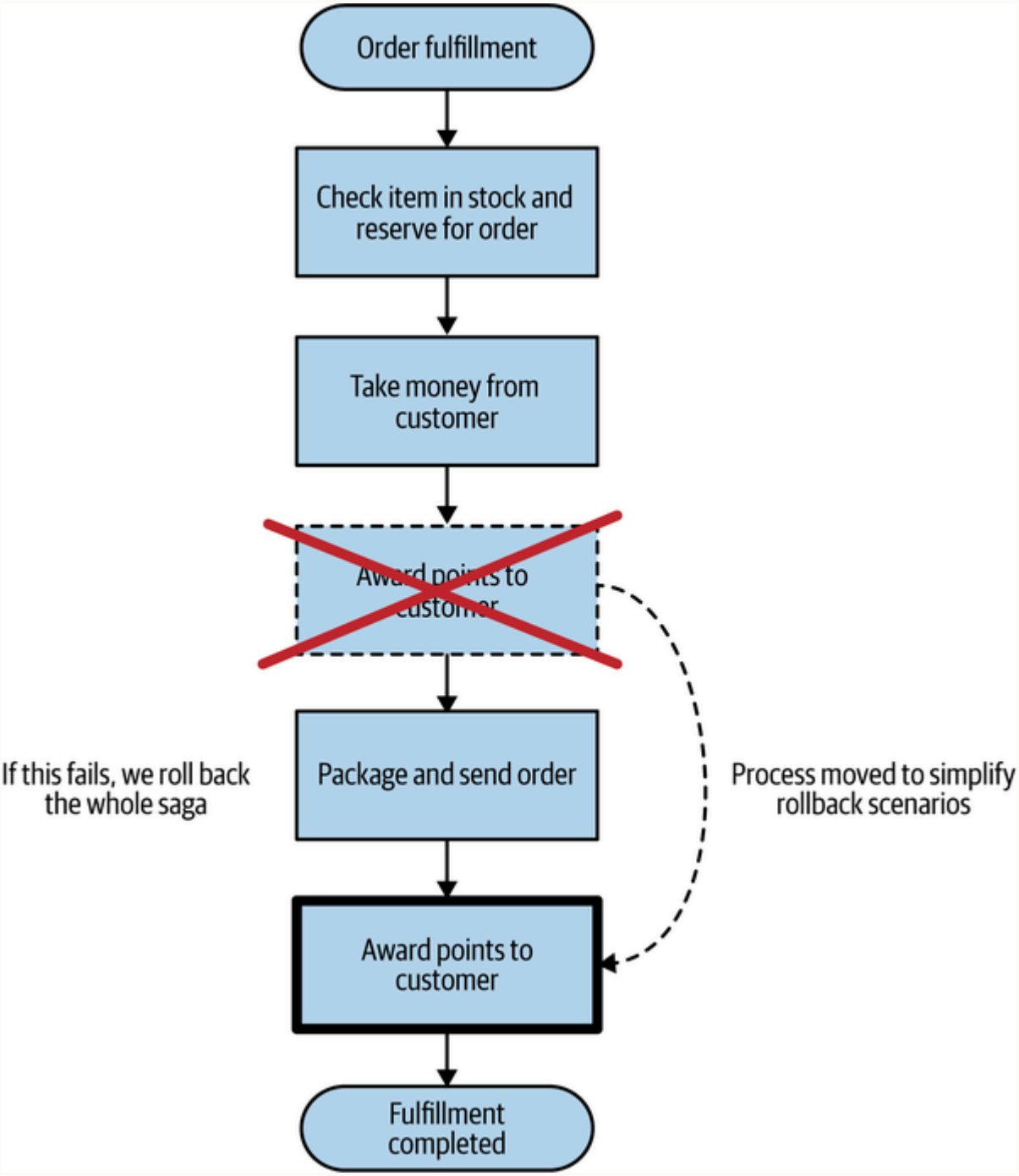


Figure 6-8. Moving steps later in the saga can reduce what has to be rolled back in case of a failure

In this way we would avoid having to worry about that stage being rolled back if we were to have a problem while trying to package and send the order. Sometimes you can simplify your rollback operations just by tweaking how your workflow is carried out. By pulling forward those steps that are most likely to fail and failing the process earlier, you avoid having to trigger later compensating transactions, as those steps weren't even triggered in the first place.

These changes, if they can be accommodated, can make your life much easier, avoiding the need to even create compensating transactions for some steps. This can be especially important if implementing a compensating transaction is difficult. You may be able to move a step later in the process to a stage at which it never needs to be rolled back.

Mixing fail-backward and fail-forward situations

It is totally appropriate to have a mix of failure recovery modes. Some failures may require a rollback (fail backward); others may be fail forward. For the order processing, for example, once we've taken money from the customer, and the item has been packaged, the only step left is to dispatch the package. If for whatever reason we can't dispatch the package (perhaps the delivery firm we use doesn't have space in its vans to take an order today), it seems very odd to roll the whole order back. Instead, we'd probably just retry the dispatch (perhaps queuing it for the following day), and if that fails, we'd require human intervention to resolve the situation.

Implementing Sagas

So far we've looked at the logical model for how sagas work, but we need to go a bit deeper to examine ways of implementing the saga itself. We can look at two styles of saga implementation. *Orchestrated sagas* more closely follow the original solution space and rely primarily on centralized coordination and tracking. These can be compared to *choreographed sagas*, which avoid the need for centralized coordination in favor of a more loosely coupled model but can make tracking the progress of a saga more complicated.

Orchestrated sagas

Orchestrated sagas use a central coordinator (what we'll call an *orchestrator* from now on) to define the order of execution and to trigger any required compensating action. You can think of orchestrated sagas as a command-and-control approach: the orchestrator controls what happens and when, and with that comes a good degree of visibility into what is happening with any given saga.

Taking the order fulfillment process in Figure 6-5, let's see how this central coordination process would work as a set of collaborating services, as shown in Figure 6-9.

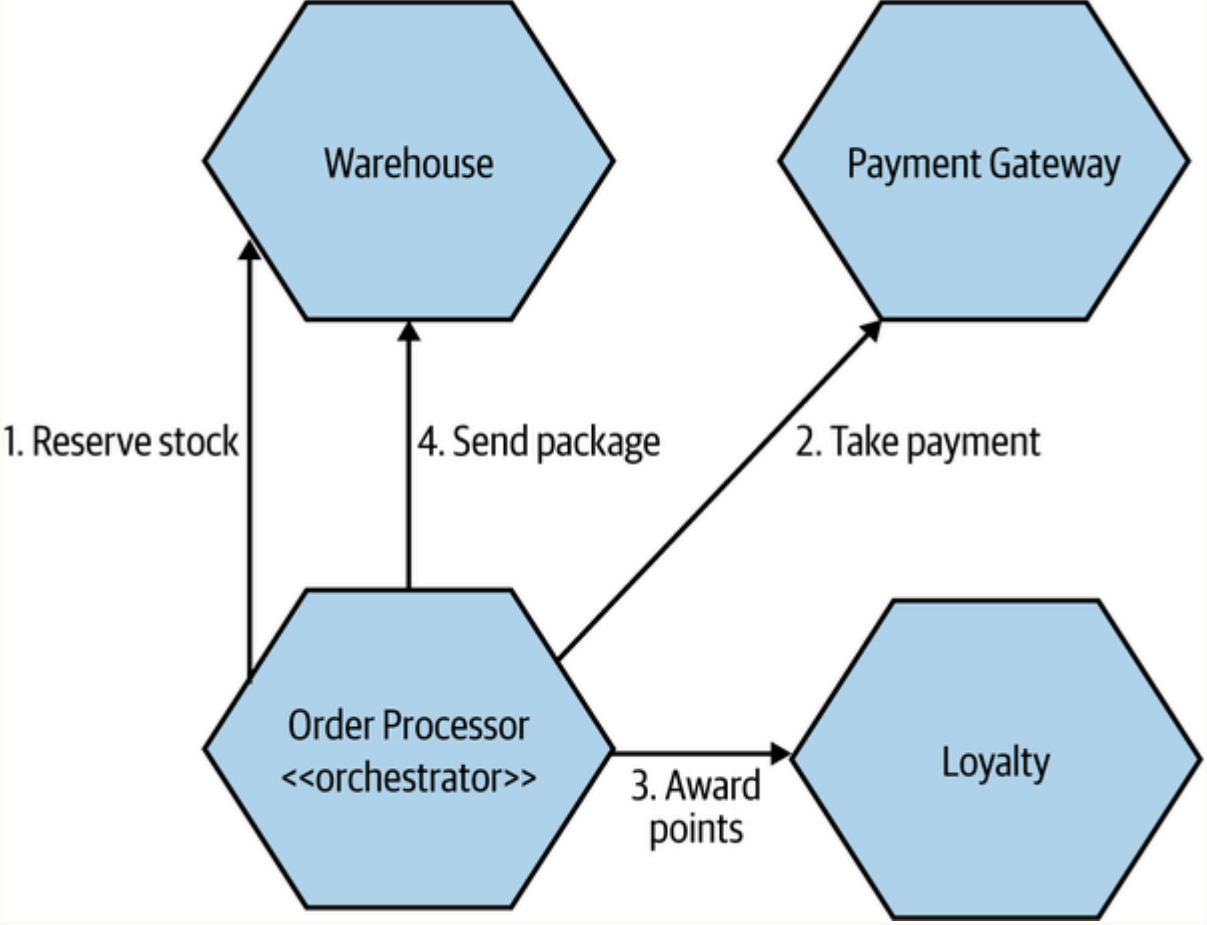


Figure 6-9. An example of how an orchestrated saga may be used to implement our order fulfillment process

Here, our central `Order Processor`, playing the role of the orchestrator, coordinates our fulfillment process. It knows what services are needed to carry out the operation, and it decides when to make calls to those services. If the calls fail, it can decide what to do as a result. In general, orchestrated sagas tend to make heavy use of request-response interactions between services: the `Order Processor` sends a request to services (such as a `Payment Gateway`) and expects a response to let it know if the request was successful and provide the results of the request.

Having our business process explicitly modeled inside the `Order Processor` is extremely beneficial. It allows us to look at one place in our system and understand how this process is supposed to work. That can make the onboarding of new people easier and help impart a better understanding of the core parts of the system.

There are a few downsides to consider, though. First, by its nature, this is a somewhat coupled approach. Our `Order Processor` needs to know about all the associated services, resulting in a higher degree of domain coupling. While domain coupling is not inherently bad, we'd still like to keep it to a minimum, if possible. Here, our `Order Processor` needs to know about and control so many things that this form of coupling is hard to avoid.

The other issue, which is more subtle, is that logic that should otherwise be pushed into the services can start to become absorbed in the orchestrator instead. If this begins to

happen, you may find that your services become anemic, with little behavior of their own, just taking orders from orchestrators like the `Order Processor`. It's important you still consider the services that make up these orchestrated flows as entities that have their own local state and behavior. They are in charge of their own local state machines.

WARNING

If logic has a place where it can be centralized, it will become centralized!

One way to avoid too much centralization with orchestrated flows is to ensure you have different services playing the role of the orchestrator for different flows. You might have an `Order Processor` microservice that handles placing an order, a `Returns` microservice to handle the return and refund process, a `Goods Receiving` microservice that handles new stock arriving and being put on the shelves, and so on. Something like our `Warehouse` microservice may be used by all those orchestrators; such a model makes it easier for you to keep functionality in the `Warehouse` microservice itself, allowing you to reuse functionality across all those flows.

BPM TOOLS

Business process modeling (BPM) tools have been available for many years. By and large, they are designed to allow nondevelopers to define business process flows, often using visual drag-and-drop tools. The idea is that developers would create the building blocks of these processes, and then nondevelopers would wire these building blocks together into the larger process flows. The use of such tools seems to line up really nicely as a way of implementing orchestrated sagas, and indeed, process orchestration is pretty much the main use case for BPM tools (or, in reverse, the use of BPM tools results in you having to adopt orchestration).

In my experience, I've come to greatly dislike BPM tools. The main reason is that the central conceit—that nondevelopers will define the business process—has in my experience almost never been true. The tooling aimed at nondevelopers ends up getting used by developers, and unfortunately these tools often work in ways that are alien to how developers like to work. They often require the use of GUIs to change the flows, the flows they create may be difficult (or impossible) to version control, the flows themselves may not be designed with testing in mind, and more.

If your developers are going to be implementing your business processes, let them use tooling that they know and understand and that is fit for their workflows. In general, this means just letting them use code to implement these things! If you need visibility into how a business process has been implemented or how it is operating, then it is far easier to project a visual representation of a workflow from code than it is to use a visual representation of your workflow to describe how your code should work.

There are efforts to create more developer-friendly BPM tools. Feedback on these tools from developers seems to be mixed, but the tools have worked well for some, and it's good to see people trying to improve on these frameworks. If you feel the need to explore these tools further, do take a look at Camunda and Zeebe, both of which are open source orchestration frameworks targeting microservice developers, and which would be at the top of my list if I decided that a BPM tool was for me.

Choreographed sagas

A choreographed saga aims to distribute responsibility for the operation of the saga among multiple collaborating services. If orchestration is a command-and-control approach, choreographed sagas represent a trust-but-verify architecture. As we'll see in our example in Figure 6-10, choreographed sagas will often make heavy use of events for collaboration between services.

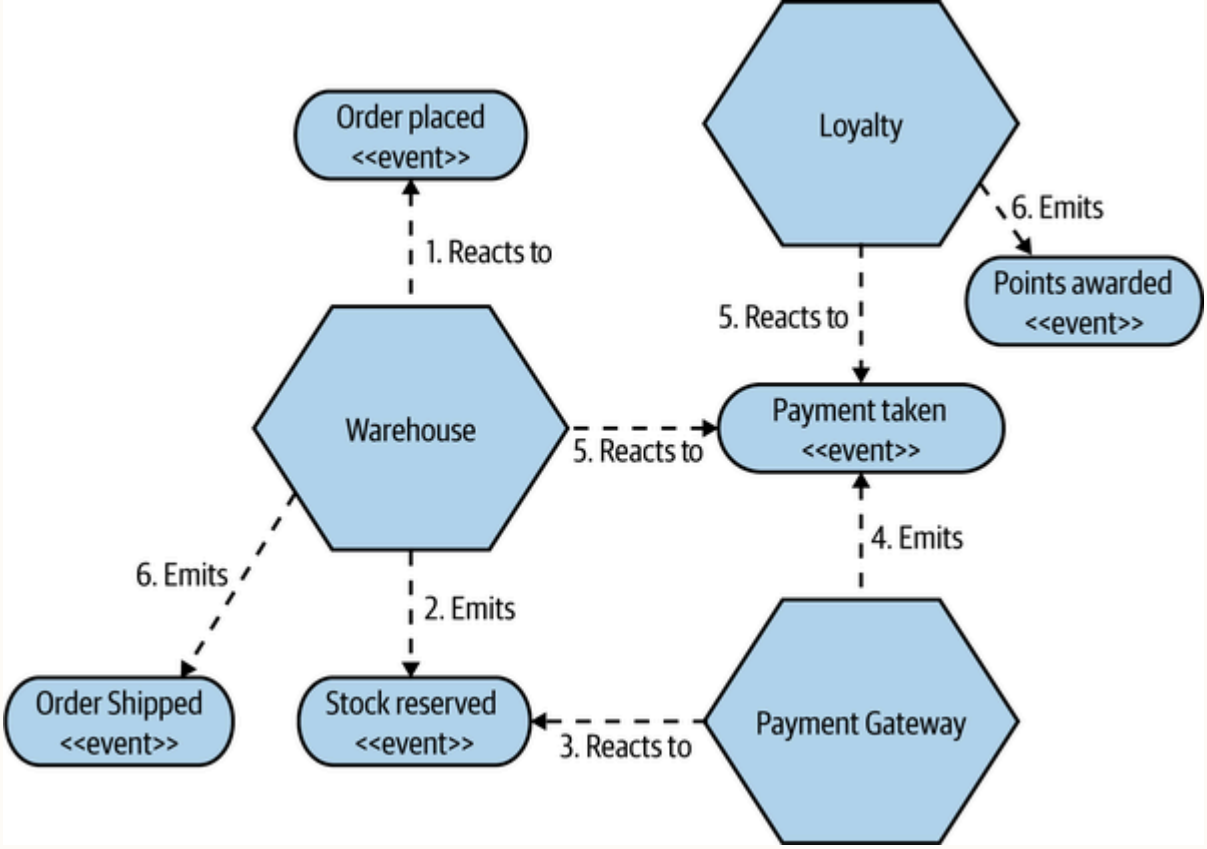


Figure 6-10. An example of a choreographed saga for implementing order fulfillment

There's quite a bit going on here, so it's worth exploring in more detail. First, these microservices are reacting to events being received. Conceptually, events are broadcast in the system, and interested parties are able to receive them. Remember, as we discussed in Chapter 4, you don't send events *to* a microservice; you just fire them out, and the microservices that are interested in these events are able to receive them and act accordingly. In our example, when the `Warehouse` service receives that first `Order Placed` event, it knows its job is to reserve the appropriate stock and fire an event once that is done. If the stock couldn't be received, the `Warehouse` would need to raise an appropriate event (an `Insufficient Stock` event, perhaps), which might lead to the order being aborted.

We also see in this example how events can facilitate parallel processing. When the `Payment Taken` event is fired by the `Payment Gateway`, it causes reactions in both the `Loyalty` and `Warehouse` microservices. The `Warehouse` reacts by dispatching the package, while the `Loyalty` microservice reacts by awarding points.

Typically, you'd use some sort of message broker to manage the reliable broadcast and delivery of events. It's possible that multiple microservices may react to the same event, and that is where you would use a topic. Parties interested in a certain type of event would subscribe to a specific topic without having to worry about where these events came from, and the broker ensures the durability of the topic and that the events on it are successfully delivered to subscribers. As an example, we might have a

`Recommendation` service that also listens to `Order Placed` events and uses that to construct a database of music choices you might like.

In the preceding architecture, no one service knows about any other microservice. They need to know only what to do when a certain event is received—we've drastically reduced the amount of domain coupling. Inherently, this makes for a much less coupled architecture. As the implementation of the process is decomposed and distributed among the three microservices here, we also avoid the concerns about centralization of logic (if you don't have a place where logic can be centralized, then it won't be centralized!).

The flip side of this is that it can be harder to work out what is going on. With orchestration, our process was explicitly modeled in our orchestrator. Now, with this architecture as it is presented, how would you build up a mental model of what the process is supposed to be? You'd have to look at the behavior of each service in isolation and reconstitute this picture in your own head—far from straightforward, even with a simple business process like this one.

The lack of an explicit representation of our business process is bad enough, but we also lack a way of knowing what state a saga is in, which can deny us the chance to attach compensating actions when required. We can push some responsibility to the individual services for carrying out compensating actions, but fundamentally we need a way of knowing what state a saga is in for some kinds of recovery. The lack of a central place to interrogate around the status of a saga is a big problem. We get that with orchestration, so how do we solve that here?

One of the easiest ways of doing this is to project a view regarding the state of a saga by consuming the events being emitted. If we generate a unique ID for the saga, what is known as a *correlation ID*, we can put it into all of the events that are emitted as part of this saga. When one of our services reacts to an event, the correlation ID is extracted and used for any local logging processes, and it's also passed downstream with any further calls or events that are fired. We could then have a service whose job it is to just vacuum up all these events and present a view of what state each order is in, and perhaps programmatically carry out actions to resolve issues as part of the fulfillment process if the other services couldn't do it themselves. I consider some form of correlation ID essential for choreographed sagas like this, but correlation IDs also have a lot of value more generally, something we explore in more depth in Chapter 10.

Mixing styles

While it may seem that orchestrated and choreographed sagas are diametrically opposing views of how sagas could be implemented, you could easily consider mixing and matching models. You may have some business processes in your system that more naturally fit one model or another. You may also have a single saga that has a mix of styles. In the order fulfillment use case, for example, inside the boundary of the `Warehouse` service, when managing the packaging and dispatch of an order, we may use an orchestrated flow even if the original request was made as part of a larger choreographed saga.[6]

If you do decide to mix styles, it's important that you still have a clear way to understand what state a saga is in, and what activities have already happened as part of a saga. Without this, understanding failure modes becomes complex, and recovery from failure is difficult.

TRACING CALLS

Whether you chose choreography or orchestration, when implementing a business process using multiple microservices, it's common to want to be able to trace all the calls related to the process. This can sometimes be just to help you understand if the business process is working correctly, or it could be to help you diagnose a problem. In Chapter 10 we'll look at concepts such as correlation IDs and log aggregation and how they can help in this regard.

Should I use choreography or orchestration (or a mix)?

The implementation of choreographed sagas can bring with it ideas that may be unfamiliar to you and your team. They typically assume heavy use of event-driven collaboration, which isn't widely understood. However, in my experience, the extra complexity associated with tracking the progress of a saga is almost always outweighed by the benefits associated with having a more loosely coupled architecture.

Stepping aside from my own personal tastes, though, the general advice I give regarding orchestration versus choreography is that I am very relaxed in the use of orchestrated sagas when one team owns implementation of the entire saga. In such a situation, the more inherently coupled architecture is much easier to manage within the team boundary. If you have multiple teams involved, I greatly prefer the more decomposed choreographed saga, as it is easier to distribute responsibility for implementing the saga to

the teams, with the more loosely coupled architecture allowing these teams to work more in isolation.

It's worth noting that as a general rule, you'll be more likely to gravitate toward request-response–based calls with orchestration, whereas choreography tends to make heavier use of events. This isn't a hard rule, just a general observation. My own general leaning toward choreography is likely a function of the fact that I tend to gravitate toward event-driven interaction models—if you are finding the use of event-driven collaboration difficult to get your head around, choreography might not be for you.

Sagas Versus Distributed Transactions

As I hope I have broken down by now, distributed transactions come with some significant challenges, and outside of some very specific situations I tend to avoid them. Pat Helland, a pioneer in distributed systems, distills the fundamental challenges that come with implementing distributed transactions for the kinds of applications we build today:[7]

> *In most distributed transaction systems, the failure of a single node causes transaction commit to stall. This in turn causes the application to get wedged. In such systems, the larger it gets, the more likely the system is going to be down. When flying an airplane that needs all of its engines to work, adding an engine reduces the availability of the airplane.*

In my experience, explicitly modeling business processes as a saga avoids many of the challenges of distributed transactions, while having the added benefit of making what might otherwise be implicitly modeled processes much more explicit and obvious to your developers. Making the core business processes of your system a first-class concept will have a host of advantages.

Summary

So as we can see, the path to implementing workflows in our microservice architecture comes down to explicitly modeling the business process we are trying to implement. This brings us back to the idea of modeling aspects of our business domain in our microservice architecture—explicitly modeling business processes makes sense if our microservice boundaries are also defined primarily in terms of our business domain.

Whether you decide to gravitate more toward orchestration or toward choreography, hopefully you're much better placed to know what model will fit your problem space

better.

If you want to explore this space in more detail, although sagas aren't *explicitly* covered, *Enterprise Integration Patterns* by Gregor Hohpe and Bobby Woolf has a number of patterns that can be incredibly useful when implementing different types of workflow.[8] I can also heartily recommend *Practical Process Automation* by Bernd Ruecker.[9] Bernd's book focuses much more on the orchestration side of sagas, but it is packed with useful information that makes it a natural following-on point for this topic.

Now we have a sense of how our microservices can communicate and coordinate with each other, but how do we build them in the first place? In our next chapter, we'll look at how to apply source control, continuous integration, and continuous delivery in the context of a microservice architecture.

1 This has changed, with support for multidocument ACID transactions being released as part of Mongo 4.0. I haven't used this feature of Mongo myself; I just know it exists!

2 Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol: O'Reilly, 2017).

3 Robert Kubis, "Google Cloud Spanner: Global Consistency at Scale," Devoxx, November 7, 2017, YouTube video, 33:22, *https://oreil.ly/XHvY5*.

4 Hector Garcia-Molina and Kenneth Salem, "Sagas," *ACM Sigmod Record* 16, no. 3 (1987): 249–59.

5 We really can't. I've tried!

6 It's outside the scope of this book, but Hector Garcia-Molina and Kenneth Salem went on to explore how multiple sagas could be "nested" to implement more complex processes. To read more on this topic, see Hector Garcia-Molina et al., "Modeling Long-Running Activities as Nested Sagas," *Data Engineering* 14, no. 1 (March 1991: 14–18).

7 See Pat Helland, "Life Beyond Distributed Transactions: An Apostate's Opinion," *acmqueue* 14, no. 5 (December 12, 2016).

8 Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

9 Bernd Ruecker, *Practical Process Automation* (Sebastopol: O'Reilly, 2021).

Previous chapter

< 5. Implementing Microservice Communication

Next chapter

7. Build >